
MIRTE MASTER-BASED LABORATORY CLEAN-UP ROBOT

AUTHORS: MATTHEW DE LANNOY, OTTO MEIJERHOF, MAX VAN 'T HART, FLORIAN VAN DEN BERG
DELFT UNIVERSITY OF TECHNOLOGY

12th Jun, 2026

Abstract

In laboratory environments, small objects often fall onto the floor and need to be collected and separated rather than discarded collectively. This thesis presents a mobile manipulation system based on the MIRTE Master platform that combines navigation, perception, and manipulation to detect and sort objects in a laboratory setting. The system integrates a 4-DOF robotic arm, a two-bin sorting mechanism, an RGB-D camera, MoveIt 2, and Nav2 with SLAM. Different coverage planners and perception approaches were evaluated in practice. The morphology-based skeleton planner performed well in the tested laboratory-like environments but worse in larger open spaces, whereas the spanning-tree planner performed well in larger open spaces but struggled more in tighter environments. The perception pipeline was able to localize and classify representative objects, and the manipulator modifications improved grasp reliability, although the system remained sensitive to lighting conditions, hardware constraints, and synchronization issues. Overall, the work shows that the MIRTE Master is a feasible platform for autonomous laboratory cleaning, while also identifying several directions for further development.

Preface

This report marks the end of our Bachelor's degree at TU Delft. Over the past few months, our team has engaged in designing an autonomous mobile laboratory-cleaning robot with a waste-separation mechanism. This has been achieved by integrating a 4-DOF manipulator, an RGBD camera and a software stack based on ROS2. We were drawn to this project because of the potential for creative freedom it seemed to offer, given that the main goal wasn't set in stone. This provided us with a hands-on learning experience that combines the theoretical and physical sides of robotics. Having a tangible robotic prototype as an end result was, as expected, a good motivator to keep on working diligently on the project. During the preparation of this paper, ChatGPT and Gemini were used as a supporting tool for brainstorming, improving sentence structure, refining argumentation, debugging, ideating, and checking the clarity of written sections.

We would like to express our sincere gratitude towards our project supervisors, Martijn Wisse and Martin Klomp for their technical insights and feedback regarding the project. In addition, we would like to thank Arend-Jan van Hilten for helping us occasionally, despite not actually being involved in the project directly.

Contents

1	Introduction	2
1.1	Related Works	2
1.2	Contribution	2
1.3	Roadmap	2
2	System Overview	2
2.1	Navigation	3
2.2	Robot Behavior	5
2.3	Mechanical	6
2.4	Manipulation	7
2.5	Perception	7
3	Implementation	11
3.1	Simulation	12
3.2	Sim-to-Real Pipeline	12
3.3	Testing Environment	12
3.4	Testing Objects	12
4	Results	12
4.1	Coverage Planners	12
4.2	Object Localization and Classification	13
4.3	Manipulation	13
5	Discussion	14
5.1	Delays	14
5.2	Gazebo	14
5.3	The MIRTE Master Platform	14
6	Conclusion	14

6.1	Future Work	14
7	Appendix	15
7.1	Algorithms	15
7.2	Materials	19
7.3	Figures	21
	References	22

1 Introduction

In laboratory environments, maintaining a good level of cleanliness is often challenging. While autonomous floor-cleaning robots are common in domestic settings, laboratory floors often contain small graspable items rather than dust or hair. A laboratory-cleaning robot therefore needs to detect, pick up, and sort such objects rather than simply sweep debris. Because not all found objects should be discarded, the system should also be able to separate reusable items, such as electronic components, from other waste. The implementation of such a system also provides useful insights into existing robotic platforms like the MIRTE Master robot.

The goal of this project was to develop an autonomous laboratory-cleaning robot based on the MIRTE Master platform. The robot should be able to navigate an indoor environment, detect objects, distinguish between categories of waste, and place them in the appropriate bin. This report outlines the development process and implementation of the resulting system.

From the main goal, the following sub-goals were extracted: adapt and enhance the design of the MIRTE Master platform, implement mapping and navigation pipelines for waste discovery, develop custom waste-detection and localization algorithms, and build a waste-processing subsystem capable of classifying, manipulating, and sorting items reliably.

1.1 Related Works

Related work on autonomous coverage planning and mobile manipulation provides the context for this project. Coverage-planning methods such as morphology-based skeleton planning and spanning-tree approaches have been proposed to generate efficient paths for inspection and cleaning tasks, but their performance depends strongly on environment geometry and map quality [1], [2]. Recent surveys of mobile-robot path planning and ROS 2 navigation also highlight the practical trade-offs between planner choice, map representation, and computational resource use in real systems [3], [4].

In parallel, research on mobile manipulation has shown that robust object perception and grasp planning are essential for pick-and-place tasks in real indoor environments [5]. The present work builds on these ideas by combining a coverage planner, a 3D perception pipeline, and a manipulator on the MIRTE Master platform to evaluate how they perform in a laboratory-cleaning scenario. In particular, the study considers object localization and classification as part of the broader task of detecting, collecting, and sorting small items.

1.2 Contribution

This work serves as a practical application of the MIRTE Master platform for a specific laboratory-cleaning task. The main contribution is to show how the platform can be extended with a waste-sorting mechanism and to evaluate which navigation and perception approaches are most suitable for this purpose. By integrating perception, navigation, and manipulation in a real-world setup, the project provides insight into the platform's practical strengths and limitations, as well as its potential for extensibility and modularity.

1.3 Roadmap

This report is structured as follows. Section 1 ([Materials](#)) describes the hardware and software used to build the robot. Section 2 ([Theory](#)) explains the background concepts behind the implemented methods. Section 3 ([System Overview](#)) presents the system architecture and the main functional modules. Section 4 ([Experimental Methods](#)) describes how the implementations were tested. Section 5 ([Results](#)) discusses the experimental outcomes. Section 6 ([Conclusion](#)) reflects on the project objective and the extent to which it was achieved.

2 System Overview

The setup consists of a mobile manipulator (MIRTE Master) tasked with autonomously exploring an indoor laboratory environment, identifying and localizing objects, distinguishing between electronics and other objects, and sorting these objects accordingly.

To realize this, the robot incorporates several distinct, but tightly knit software subsystems. Each one is responsible for a separate function of autonomous operation, but simultaneously reliant on data from one or more of the other systems.

At the highest level sits a behavior tree that orchestrates how these different subsystems interact to make the robot execute the main goal. It decides when to explore, when to approach a detected object, when to classify it, and when to pick and place it.

Below this, a navigation system handles how the robot moves through the environment. It maps, then plans a coverage path, enabling the discovery of portable objects. The manipulation system plans and executes pick and place motions given object poses and visibility. Underpinning both is the perception pipeline, which provides the system with a continuous stream of object poses and classifications derived from depth and colour data.

These subsystems do not operate in isolation. The behavior tree depends on perception to make decisions, perception depends on navigation to bring the camera within range of objects, and manipulation de-

depends on the localization estimates that perception provides. Understanding the system therefore requires understanding how these parts interact, not just what each part does individually.

The following sections describe each subsystem in turn: behavior and task orchestration, manipulation and motion planning, perception, and navigation.

2.1 Navigation

The navigation stack of this robot is responsible for two main tasks: mapping and coverage. During mapping, the robot builds a representation of the environment from sensor data so that it can plan safe trajectories. During coverage, it generates a sequence of waypoints that traverses the free space while avoiding obstacles.

2.1.1 Mapping

Before the robot can navigate through the environment and ensure proper coverage, the environment must first be known. This is where mapping approaches become useful. Several advanced and specialized mapping approaches already exist, but in this work only one was considered: frontier-based mapping. Frontier-based mapping allows the robot to start navigating the environment and expand its map as new areas are discovered, which is well suited to partially known indoor spaces [6]. For the actual motion execution, Nav2 is used as the low-level navigation framework. The coverage planners implemented in this work generate the waypoints that define the desired path, while Nav2 handles trajectory generation, obstacle avoidance, and execution of the motion in the live costmap [7]. This separation keeps the higher-level planning logic simple while allowing the robot to respond to dynamic obstacles in real time. A video demonstration of the robot mapping a space is available [online](#).

2.1.2 Coverage Planning Algorithms

This section presents the algorithms employed for the discovery of portable objects. While a comprehensive technical analysis of these algorithms is beyond the scope of this work, they are described with sufficient detail to enable understanding of their application within our framework. For in-depth technical details and formal derivations, the reader is referred to the original publications cited herein. To demonstrate the different navigators, a costmap of the Pulse Breakout room is used. This map was obtained during testing of the coverage methods. Furthermore, Nav2 [7] is used for low-level motion planning and execution as well as real-time control and costmap generation.

Preprocessing Before the map can be used to plan a path, a common preprocessing algorithm can be abstracted. Coverage path planners generally use either a binary map of the free space (free = 1, occupied =

0) or the polygons constructed from the boundaries of the free space.

The input to the algorithm can be any 2D image displaying the free and occupied pixels. In this case, the input is the costmap directly. First, the costmap is thresholded to create a binary costmap (Figure 1), which is the first output.



Figure 1: Binary costmap generated by thresholding the Nav2 costmap into free and occupied space.

Then, using this binary costmap, the contours are extracted (Figure 2).

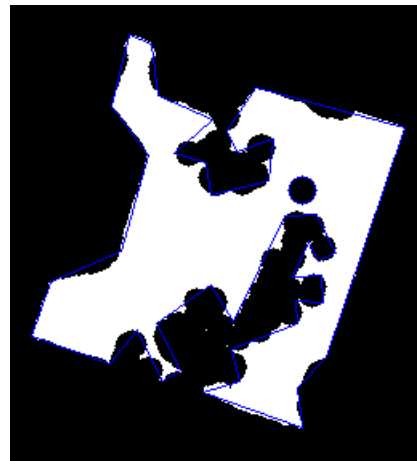


Figure 2: Contours extracted from the binary costmap representing the boundaries of free space.

Morphology-Based Skeleton Path Planner The first implemented method, based on the work of [1], [8], derives a coverage path from the morphological skeleton of the free space. The algorithm is visible in the Appendix.

The free space is extracted using the methods discussed in the previous section. The polygons are used and converted to a binary map so as not to produce too many branches for the robot to navigate through.

This map is then skeletonised to extract waypoints, see Figure 3.

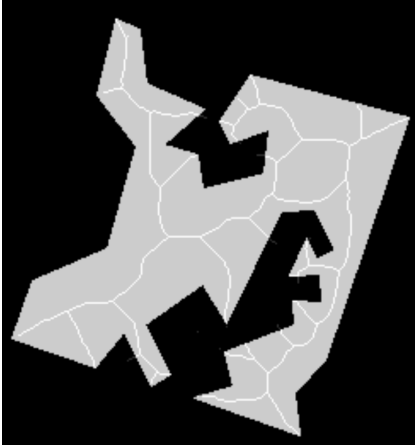


Figure 3: Morphological skeleton extracted from the free-space representation of the environment.

Once the waypoints are obtained, a graph is constructed and leaf nodes are extracted. In the graph, leaf nodes are defined by their single connection to the rest of the skeleton; these are treated as endpoints of the path.

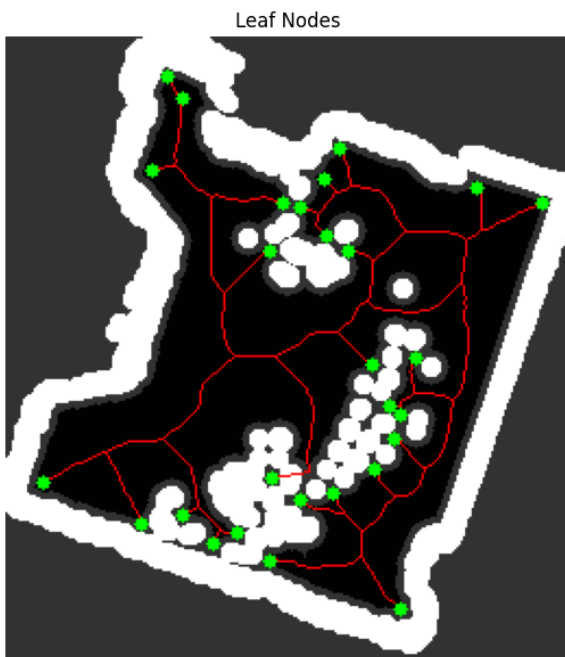


Figure 4: Leaf nodes identified on the skeleton graph, representing endpoints used during path generation.

Finally, starting from the leaf node closest to the robot, the full path is planned by recursively tracing the graph from the current leaf node to the nearest unvisited leaf node. This can be seen in Figure 5.

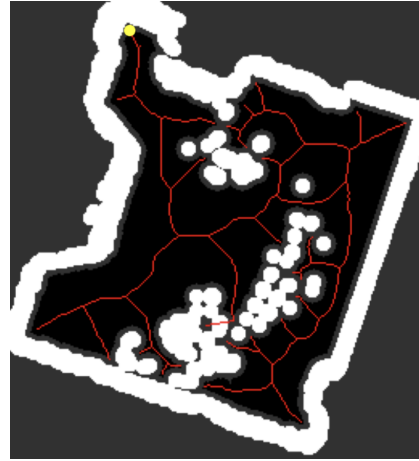


Figure 5: Fully planned path

Grid-Based Spanning-Tree Planner Secondly, a grid-based spanning-tree algorithm is used to plan a full coverage path over the map. This planner is based on the work of [2].

Unlike the skeleton planner, this planner uses the binary costmap directly. The binary costmap is further discretized to a grid whose resolution is a fraction of that of the original costmap, see Figure 6.

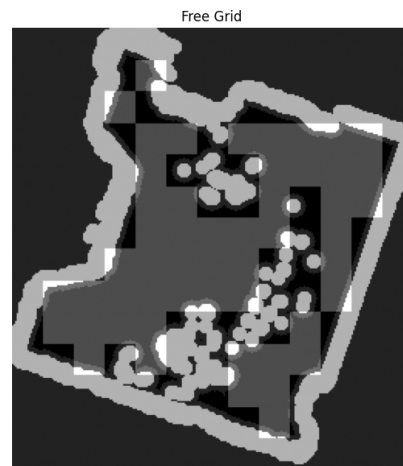


Figure 6: Downsampled free-space grid used to construct the spanning tree graph.

A depth-first search (DFS) is conducted over the free cells, from which sparse waypoints are created. This can be seen in Figure 7.

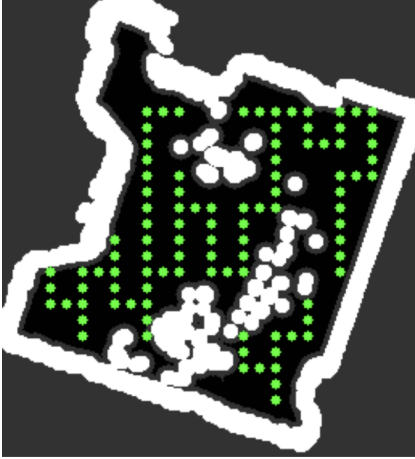


Figure 7: Depth-first search spanning tree generated over the free-space grid.

A black image with the same resolution as the original costmap is then created, and white squares with half the cell width are placed in the center of each cell and between adjacent cells. Waypoints are created from the contours of this tree. These waypoints represent a circumnavigation around the DFS tree.

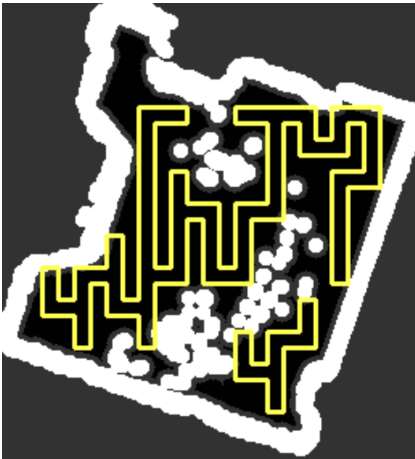


Figure 8: Coverage waypoints generated by tracing the contour around the spanning tree structure.

A segment from this circumnavigation is then removed from the path.

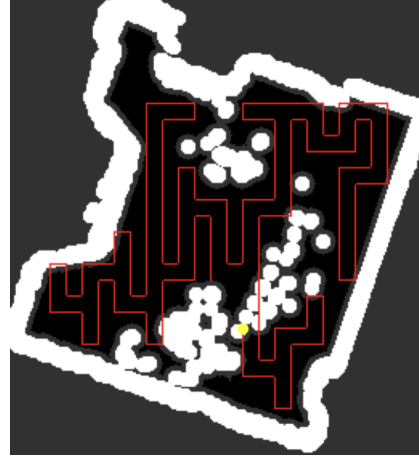


Figure 9: The path with circumnavigation removed.

A lab cleanup robot must not blindly follow waypoints. If it does, it might collide with obstacles near the map boundary or collide with moving objects.

Nav2 implements several low-level real-time trajectory planners that plan inside the live-updating costmap [4]. They handle how the waypoints are followed and how much influence the map ought to have on the generated trajectory. The system described in this section implements an MPPI controller [9] due to its superior performance in highly dynamic environments [10], such as labs where students or other people frequently walk. Other planners that were considered include DWB (Dynamic Window Approach Behavior-based) [11] and RPP (Regulated Pure Pursuit) [12].

2.2 Robot Behavior

A common approach in robotics for navigation- and perception-heavy tasks is to use a global behavior tree that describes the robot's actions in different situations. This architecture, as shown in Figure 10, is used here to describe and execute the cleaning strategy. The *Py Trees for ROS* Python package is used to implement the behavior tree because of its thorough documentation and its native support in ROS 2.

The behavior tree provides a hierarchical approach for coordinating navigation, perception, and cleaning actions. It also provides a clear structure for debugging. As seen in Figure 10, the robot first explores the environment, then pauses its coverage task whenever an object is detected, approaches the object, checks whether it is still visible, picks it up, and places it in the appropriate bin. The complete tree is shown in Figure 28 in the Appendix, and the execution of the coverage task, including pausing and resuming, is described in Algorithm 3.

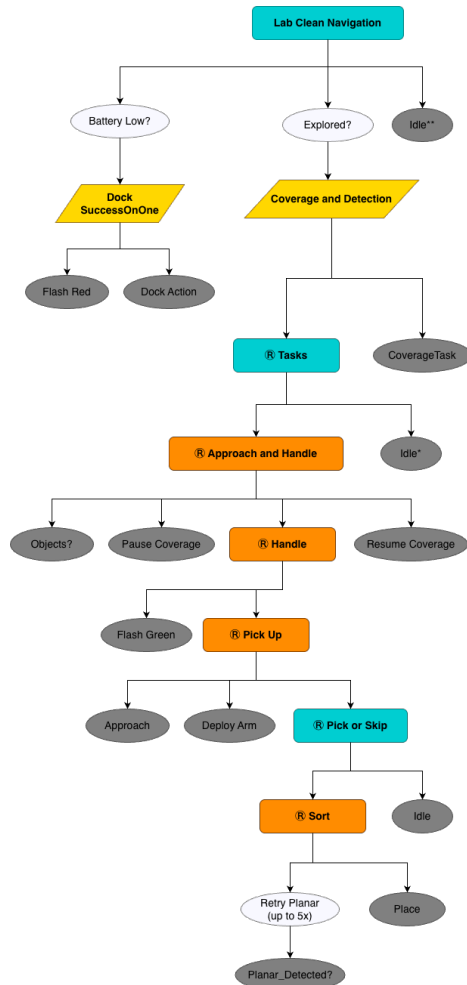


Figure 10: Behavior tree (simplified)

2.3 Mechanical

The MIRTE Master platform provides a capable but general-purpose starting point for a laboratory cleaning robot. Out of the box, it offers omnidirectional mobility, a four-degree-of-freedom manipulator, and an integrated sensor suite. However, several of its design choices were not well-suited to the specific demands of autonomous waste collection and sorting. This section documents the physical modifications made to the platform: what was changed, why, and how each modification contributes to the overall system's ability to reliably detect, grasp, and sort laboratory objects.

2.3.1 Original MIRTE Master Platform

The original MIRTE Master hardware consists of three main parts:

1. **Mobile base**
2. **Electronics**
3. **Robotic arm**

The mobile base utilizes four geared DC motors with Hall encoders, connected to 97 mm mecanum wheels.

This configuration enables omnidirectional movement, which is useful for navigating in small indoor spaces. The base also contains rearward-facing ultrasonic sensors, an IMU, a 2D LiDAR and an RGBD (depth) camera.

The electronics of the original platform are based on an Orange Pi 3B and a Raspberry Pi Pico H, following a two-tier software architecture that separates high-level task logic from low-level execution. The Orange Pi functions as the main computer, handling computationally intensive tasks. The Pico, as the embedded controller, handles lower-level control tasks such as driving the motors and reading lightweight sensor data. The MIRTE Master also includes a custom PCB to connect the motors, sensors, and power system in a structured manner, powered by a 5 Ah 12 V Li-ion Parkside drill battery. No significant changes were made to the electronics for this project.

The robotic arm is mounted on top of the mobile base and has four degrees of freedom, which are driven by four serial bus servo motors of varying strengths. A gripper and RGBD camera are mounted at the end of the arm for object scanning and manipulation.

The original hardware components can be seen in [Materials](#).

2.3.2 Design Goals for the Modified Robot

Despite being a capable platform, several aspects of the original design presented practical limitations for this specific use case. The PMMA chassis plates proved brittle under load, cracking during early testing. The RGBD camera, mounted low at the front of the chassis, was too close to the ground to produce reliable depth estimates for small objects on the floor. The gripper's contact surfaces lacked the friction needed to reliably grasp small, smooth electronic components. Finally, the platform had no provision for storing or separating collected objects. These shortcomings directly motivated the modifications described in the following sections.

The main design goals are:

- Increase the stiffness and reliability of the chassis.
- Add space for collecting and separating waste.
- Mount the camera in a position that enables object detection.
- Improve the battery mounting system.
- Improve cable management.
- Modify the gripper so it can pick up waste objects more reliably.

2.3.3 Chassis Modifications

The original MIRTE Master chassis was designed as a general-purpose mobile manipulator base. For this use case, the robot also needed to carry waste bins.

The chassis consists of six side panels and a top and a bottom plate. In the original model, the top and bottom plates were made of clear PMMA. Although PMMA allows the user to see the status of the electronics by looking at the LEDs on the inside, early testing of the MIRTE Master showed that it was not suitable, as attempts to assemble the robot often resulted in shattered plates. The fracture toughness of PMMA ($0.7\text{--}1.1 \text{ MPa}\cdot\text{m}^{1/2}$), which is approximately 30 times lower than that of aluminium ($23\text{--}35 \text{ MPa}\cdot\text{m}^{1/2}$), makes it prone to sudden brittle fracture under mechanical load. Therefore, the PMMA was replaced with aluminium. To still be able to see the status LEDs, the six side panels were 3D-printed using translucent PETG. Two separate waste bins were also added to the rear of the chassis, as they allow the robot to collect, store and separate objects after picking them up.

2.3.4 Gripper Modifications

The original MIRTE Master gripper is useful as a general robotic gripper, but the cleanup task requires it to reliably handle objects that vary widely in shape and size.

The original gripper geometry is suitable for holding small and large objects; however, the gripping surface is not. Electronic components tend to have small areas of contact with the gripper, making slippage likely during lifting. To improve gripping performance, the contact patches are layered with soft 83A TPE, coated with an additional layer of Bison Rubber anti-slip coating.

During the initial testing phase of the RGBD camera, it was found that the mounting point was too low for any objects close to the ground to be visible. Although the camera was originally intended to be mounted on the chassis, mounting it on the end of the arm proved beneficial for this project. In addition to the elevated vantage point, this allows for programmable viewing angles. Furthermore, the RGBD camera is now also used for object detection, making the secondary RGB camera unnecessary.

2.3.5 Current Design

The complete model can be explored using the interactive 3D interface on the [Mechanical](#) page of the online report. The CAD files can be found [here](#).

2.4 Manipulation

This section covers the articulation of the robotic arm that sits atop the MIRTE Master. Since there is hardly any competition in this space, the market leader,

Movelt 2, was used to move the arm. Movelt 2 provides a well-documented API and directs the vast majority of computations regarding (inverse) kinematics of robotic arms and the trajectory planning thereafter [13].

The package is already installed on the MIRTE Master and has previously been used, with limited success, to move MIRTE's arm [14]. However, this implementation resulted in the end effector (the tip of the arm; the wrist) being positioned more than 10 cm off-target. For a machine needing to pick up objects smaller than 6 cm, this is unacceptable. Therefore, an improved version is required.

2.5 Perception

Perception is responsible for object detection, classification, and spatial localization. In this study, two object-detection and localization approaches were evaluated: a **2D detection and depth-projection** approach and a 3D segmentation approach based on **point-cloud clustering**. For classification, a classical computer-vision approach and a model-based approach were both considered.

Initially, the goal was to distinguish between graspable and non-graspable objects and between electronics and non-electronics. However, because the standard MIRTE Master platform had limited camera quality and limited computational resources, the final scope was reduced to two practical classes: graspable versus non-graspable and colourful versus greyscale. This choice was made to keep the system feasible within the available hardware and to focus the evaluation on the most relevant objects for the laboratory-cleaning task.

A custom YOLO-based detector was trained on a small labelled dataset collected from the onboard camera, while the 3D pipeline used point-cloud filtering, plane removal, and clustering to estimate the position and orientation of candidate objects. The combination of these two perception streams was then used to support the grasping and sorting process.

2.5.1 Perception 2D

For determining the type, and possibly the distance of objects, a YOLO26 ('You Only Look Once-26') object-detection model was used. A dataset was created by recording videos using the USB camera included in the MIRTE Master hardware package, from which one in every fifteen frames was extracted for the training dataset. A figure of four frames in different environments is shown below in Figure 11.



Figure 11: Picture of four individual frames



Figure 12: The tests performed during training.

Why YOLO26 Was Used The YOLO object-detection family is well established, with YOLO26 [15] being the most recent generation. YOLO26 was chosen because the robot has to detect and classify objects from a live camera on the MIRTE Master platform, which has limited processing power because it uses an Orange Pi 3B. Object detection with limited processing power requires a lightweight model with fast inference times while still maintaining accuracy. YOLO26 is suitable for this because it was designed with deployment on low-power devices in mind. It uses NMS-free inference in its post-processing pipeline, simplifies bounding-box predictions, and supports deployment formats commonly used for low-power devices [16].

In this case, the RK3566 NPU of the Orange Pi 3B can be used to improve performance by reducing the total inference time [17]. For this model, the Nano variant of YOLO26 was chosen to minimize inference times. Another benefit of YOLO26 over the older variants is its support for oriented bounding boxes. Unlike standard bounding boxes, oriented bounding boxes also estimate the rotation of recognized objects [18]. This could be useful for complex grasping tasks where the orientation of the gripper is crucial. However, this project only uses standard object detection.

Training The training data was annotated manually using Label Studio. Initially, roughly 280 frames were captured in four different environments. Due to a large amount of blurring within the images, about 100 pictures were deleted. Of the remaining 180 pictures, 60 were imported into Label Studio and labelled per colour: green, red, purple, white, light-grey, dark-grey and black. In total, 1206 bounding boxes were drawn. A test of the perception model performed during training is shown in the figure below.

The curves in the figure below show that the model converged during training. This means that the model actually learns while training instead of making random or unstable decisions.

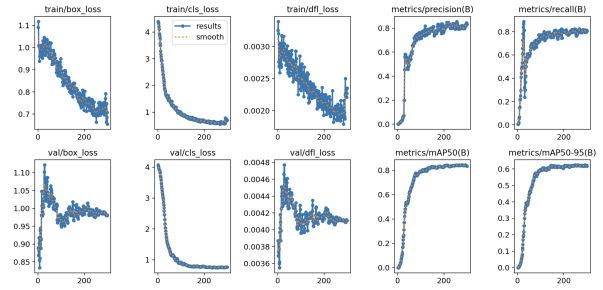


Figure 13: Plots of training results per epoch

The normalized confusion matrix shows how well classes are recognized by the model. As shown in the figure below, most classes are classified correctly, as indicated by the strong diagonal values. However, light-grey objects were among the least correctly identified objects, because changes in exposure and lighting often caused the model to classify them as background. Some objects were missed (classified as background), a small number of objects were misclassified, and some objects, such as dark-grey and white ones, were confused with light-grey.



Figure 14: Normalised confusion matrix

A figure of the F1 confidence curve is shown below. This curve shows how the F1 metric, which combines precision (how many objects were correctly predicted) and recall (how many objects were correctly found), changes with the confidence threshold of the model. The curve shows that a good starting confidence threshold would be 0.3–0.35.

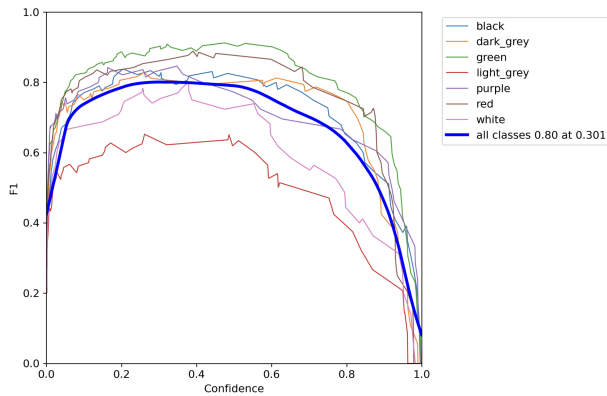


Figure 15: BoxF1 curve

2.5.2 Perception 3D

Point Cloud-Based Object Localization To determine the exact location of graspable objects around the MIRTE Master, the data gathered and transmitted by the camera must be interpreted. One effective method for achieving this is through the use of point clouds.

As the name suggests, point clouds are collections of points that represent a three-dimensional space. These points may contain a variety of information, but the most important characteristic of a point during this project is its position, represented by its x , y , and z coordinates. During this project, the `Open3D` library was used to process the point cloud. This particular

library has been selected because of its user-friendly API and popularity over PCL [19].

Planes are detected using the Random Sample Consensus (RANSAC) algorithm. This algorithm picks a number of points from the point cloud at random and fits a plane through them. It then evaluates which points in the point cloud are within the given distance from the plane [20]. With this method, point clouds can easily and quickly be filtered such that only the important parts, namely the clusters of points representing small objects on the ground, remain.

Next, a clustering algorithm called DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is used. The algorithm evaluates the remaining points in the point cloud and determines the cluster to which they belong [21]. Afterwards, the identified clusters can be converted into bounding boxes, determining an object's position and orientation, as well as its dimensions. Below, Figure 16 and Figure 17 show the input and output of the point cloud pipeline.

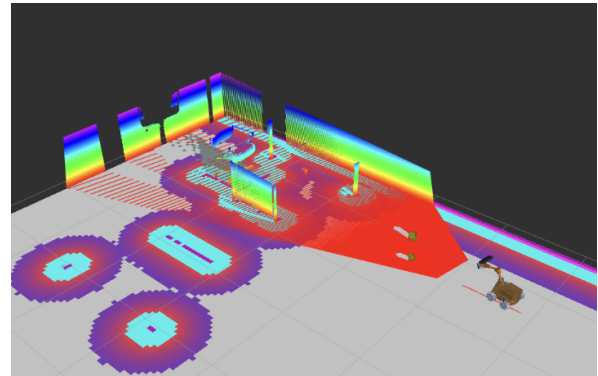


Figure 16: The point cloud as received from the camera topic

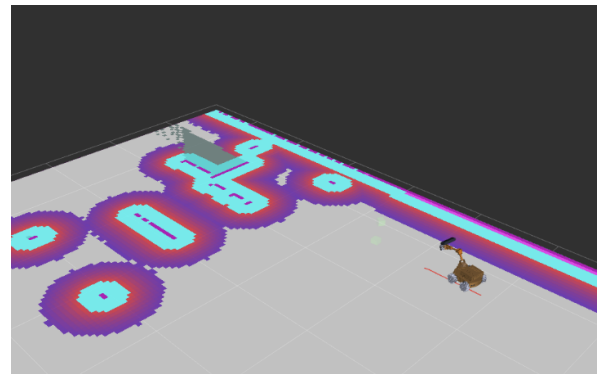


Figure 17: The resulting bounding boxes

Any objects that are part of the costmap are excluded, leaving only these bounding boxes.

2.5.3 Complete Point Cloud Processing Pipeline

This section shows the practices employed to process the point cloud received from the camera topic `/camera/depth/points`. Even though the inner workings of the specific functions used are outside the scope of this project, their influence is highlighted here to show their importance.

For completeness, a point cloud is included below to give an impression of what the input of the pipeline looks like as received from the point cloud topic.

To do this, a sample point cloud has been saved from the Gazebo simulation and is used as the input to a filtering pipeline built with the Open3D Python library:

- steps:
 - Crop the point cloud to include only points below a given height.
 - Copy a downsampled version of the point cloud to speed up future calculations.
 - Iteratively detect planes and remove the corresponding points from both point clouds. From there, the downsampled point cloud will not be used anymore.
 - Detect clusters from the remaining points.
 - Create bounding boxes for these clusters.
 - Reject boxes that overlap with the costmap.

Any remaining bounding boxes are published as detected objects.

The entire pipeline, including the individual effect of each step, is shown below:

First, the original point cloud is created from the camera's depth image, as shown in Figure 18.

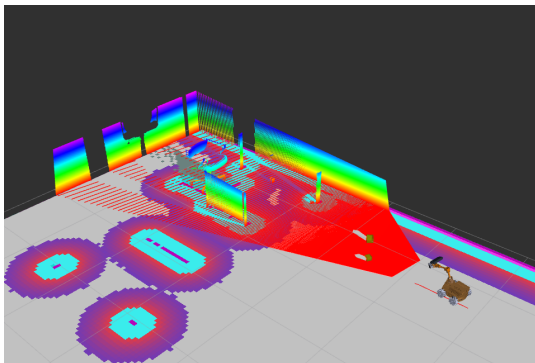


Figure 18: The point cloud as received from the camera topic including the eventual bounding boxes

Next, this point cloud is downsampled. Note that the resolution afterwards is still quite high. This is because the objects are very small, and reducing the resolution dramatically would lose details of these objects,

reducing bounding-box accuracy. An example is shown below in Figure 19.

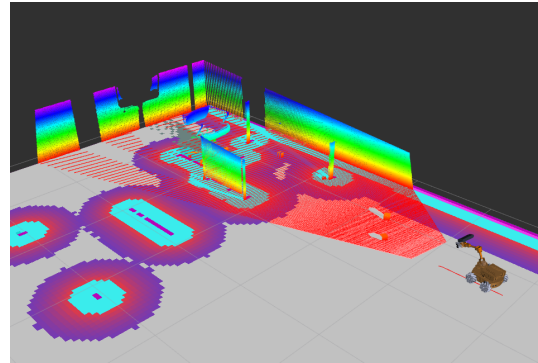


Figure 19: The downsampled point cloud

In the downsampled point cloud, planes are detected using RANSAC plane segmentation [20]. With this, large planes such as walls and tabletops are discarded from the point cloud, as can be seen in Figure 20.

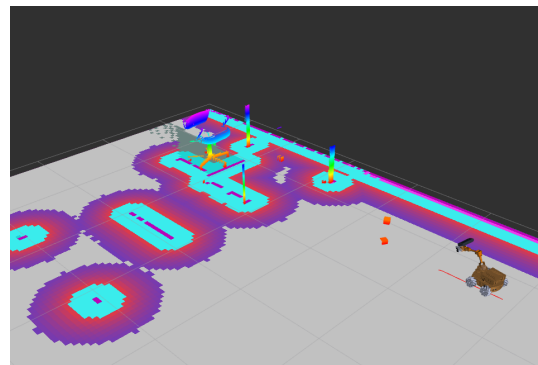


Figure 20: The plane-segmented point cloud

Now, the remaining points are cross-referenced with the costmap obtained during mapping. Any point that lies on the costmap is excluded from the point cloud, resulting in the point cloud depicted in Figure 21.

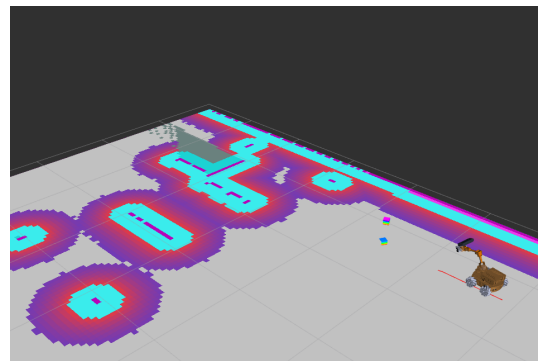


Figure 21: The point cloud containing only points belonging to free objects

Finally, the remaining points are converted to clusters using DBSCAN [21]. This enables the construction of

a bounding box for each cluster, each of which represents an object. Bounding boxes, however faint, can be made out in Figure 22 below.

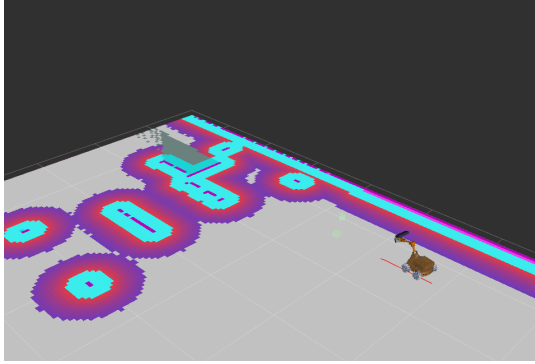


Figure 22: Bounding boxes (in front of the robot) are drawn around the remaining clusters.

Information about these bounding boxes is then published on a ROS 2 topic, which is later used to approach and pick up the corresponding objects.

3 Implementation

This section describes the transition from simulation-based development to real-world testing on the MIRTE Master platform and the practical requirements involved in doing so.

To compare the implemented approaches, the study focused on practical performance indicators: whether the robot could complete a coverage run without losing track of its pose, whether objects could be detected and localized reliably, and whether manipulation could be executed accurately enough to place objects in the correct bin. These measures were used to evaluate the different planners and perception components.

The programs required for lab cleanup operation are started from two launch files; the first launches the navigation, perception and manipulation stacks, while the second runs the behavior tree. This setup allows different behavior trees to be tested without restart-

ing the other subsystems. A brief description of each subsystem's implementation is given below.

Coverage Navigation The coverage navigation system is implemented as an action server, meaning it plans and executes the coverage path in response to a request from an external client. The request specifies which coverage planner to use. As described in Section 2.1.2, the coverage task is executed as a sequence of waypoint segments. During execution, the system continuously monitors the Nav2 action state and publishes progress feedback. If a stop or cancel request is issued, the current navigation task is interrupted and the run ends. If a pause request is issued instead, the active segment is interrupted, the remaining waypoints are stored, and the unfinished portion is re-queued so that the task can resume from the current location once the pause is lifted.

Manipulation The manipulation system also consists of an action server, accepting goal requests from the behavior tree. It exposes a single action interface that handles named pose targets, Cartesian pose goals and gripper commands. For Cartesian goals, the server runs an IK solve against the wrist link, falls back to approximate IK if exact IK fails, applies time-optimal trajectory generation, and executes the resulting plan via MoveGroupInterface.

Perception The perception pipeline consists of two independent nodes that run continuously alongside the behavior tree. The depth-based locator publishes detected object poses, which the behavior tree listens to via the blackboard. When the tree detects that objects are present, it pauses coverage navigation and approaches the closest detected object. Once the robot is within range, the 2D classifier is invoked via a service call to confirm the object's class before the pick-and-place action is triggered. The classifier's "target" or "waste" label is then passed directly to the manipulation system to determine which bin the object is placed in.

Figure 23 is a graphical overview of how the different subsystems interact.

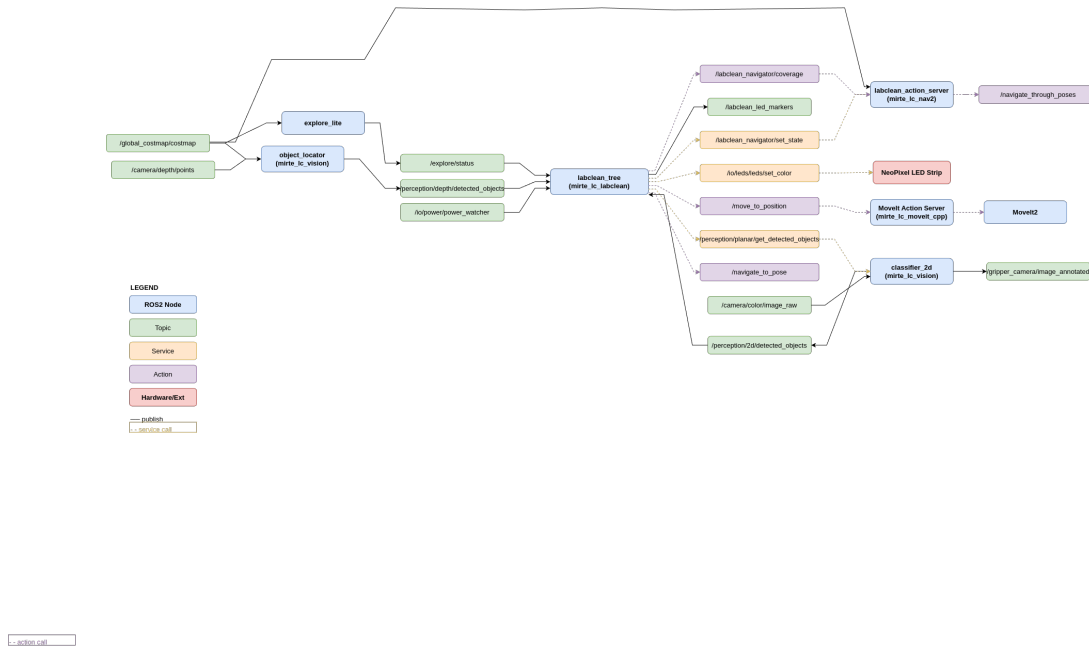


Figure 23: ROS 2 architecture.

3.1 Simulation

Development began without access to a physical robot. A Gazebo simulation environment was therefore set up to allow the software stack to be developed and tested in the interim. The simulated environment can be explored interactively in the online version of this report.

3.2 Sim-to-Real Pipeline

For the transition from simulation towards real-world testing, the codebase needed to be adapted. The launch files were modified so that Gazebo was no longer used and `use_sim_time` parameters were set to `false`. To reduce the risk of collisions, the inflation radius and the robot radius in the Nav2 parameters were increased. Another change was the migration of processing tasks from the Orange Pi 3B to a laptop connected to the MIRTE. Finally, all the sensors were tested and it was concluded that the depth camera was not positioned correctly, so its position was changed from the front of the robot to the gripper, allowing its viewing angle and distance to the object to be adjusted more effectively.

3.3 Testing Environment

For testing, an open, medium-sized room was chosen. The room needed to have sufficiently large open spaces for the robot to be able to plan its trajectories for the object-detection phase. Due to the use of mecanum wheels and camera-based object detection, the room also needed to have a mostly smooth, flat floor with a uniform colour.

3.4 Testing Objects

Due to the low video output quality of the included USB camera module, including significant motion blur and difficulties with exposure under different lighting conditions, the success rate of recognition and classification of electronic objects was too low. Additionally, most electronic waste had too low a profile to be recognized reliably by the depth camera. As a result, it was decided to 3D-print coloured shapes with a textured outer wall for grip. A custom dataset was created for the object-classification model to improve object-detection performance.

4 Results

This section covers the observed characteristics of the implemented coverage planners, perception methods and solutions regarding manipulation.

4.1 Coverage Planners

Three path planners have been compared:

- Boustrophedon Coverage
- Morphology-based Skeleton Coverage
- Spanning Tree Coverage

The Boustrophedon coverage planner package had compatibility issues with other packages used for navigation, which meant that Boustrophedon did not work on the MIRTE Master.

The morphology-based skeleton coverage planner performed well in laboratory-like environments but worse

in larger open spaces. This planner allowed the robot to maneuver efficiently between objects such as tables and chairs while still being able to get into tight places.

The morphology-based skeleton coverage planner also resulted in the fewest sharp turns, which, given the simple odometry of the MIRTE Master (wheel encoders only), helped the robot keep track of its position and thus keep the map clean. However, the skeleton planner performed slightly worse in terms of coverage performance. This was due to the lower density of the planned path, which sometimes caused the robot to miss the edges of larger free spaces that needed to be scanned for objects.

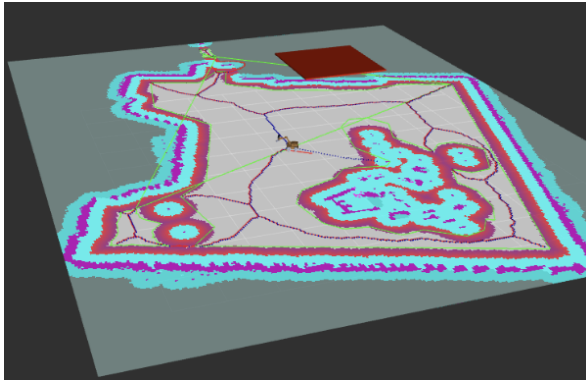


Figure 24: Skeleton path in laboratory environment

The spanning tree coverage planner performed well in large open spaces, but unlike the morphology-based skeleton coverage planner, it struggled more with planning paths in tighter spaces such as laboratory environments (see Figure 25), resulting in separate path loops being stitched together and, possibly, tighter spots between the loops being missed. Also, because of the nature of the coverage pattern, many small-radius turns are generated, which sometimes caused the robot to slowly accumulate drift in the odometry, resulting in less accurate localization and a less accurate map.

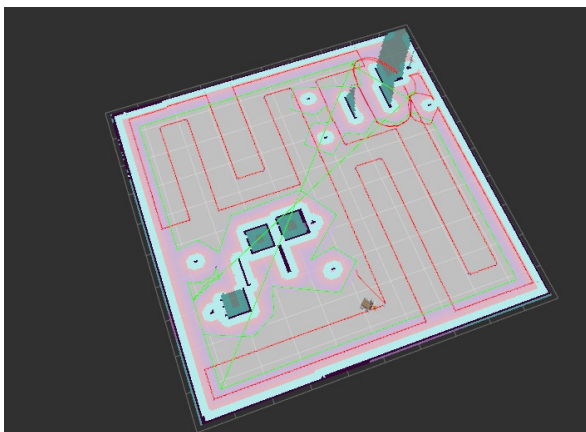


Figure 25: Spanning tree path in laboratory environment

4.2 Object Localization and Classification

For object detection, a localization method and a classification method were developed. Our object localizer relies on bounding boxes that are retrieved from point clouds generated from the depth camera's depth images. Our object detector was able to find objects sized from 1 cm to 6 cm. A bounding box drawn around a found object in the point cloud is shown in Figure 26. The custom-made YOLO26 object-detection model worked as intended, correctly classifying the localized objects. Figure 26 also shows a frame from the RGB sensor of the depth camera in the bottom-right corner.

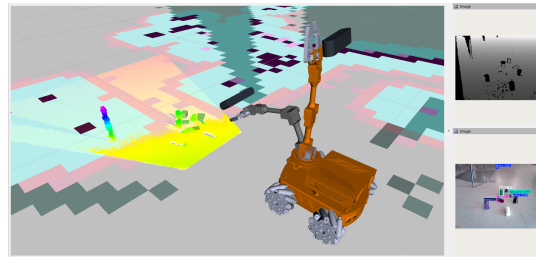


Figure 26: Bounding box drawn around found objects (center) and YOLO26 working simultaneously (bottom right)

4.3 Manipulation

Movement accuracy The manipulation of the actuator has seen major improvements regarding movement accuracy. The initial accuracy of the actuator was around 15 cm. After the code was reworked, the actuator was controlled by actual joint values instead of approximations. This reduced the error significantly, resulting in a movement error of only 1 mm.

Gripping performance The gripping performance of the original robot was slightly improved by adding rubberized patches to the gripping surfaces of the gripper. This has significantly helped the gripper hold the smooth, plastic testing objects when it approached them at a sub-optimal angle.



Figure 27: Rubberized gripping surfaces.

5 Discussion

5.1 Delays

We were caught in the middle of a platform migration to a new design. This migration happened at the time that a master's course started, which used up all of the available MIRTE Master robots. This meant that, for us, no robot was available from the start of the project. We also had no clarity about when a unit would be received and whether additional components needed to be ordered, and we could not order all components ourselves because of budget restrictions. In the end, the components were received in week 13 of the 16-week project period. While the software team was able to front-load much of the development using Gazebo simulations, this still meant that the team had only three weeks to discover the quirks of the MIRTE Master, troubleshoot them, and integrate the software, which severely limited our ability to perform real-world quantitative and qualitative experiments.

5.2 Gazebo

Gazebo was a useful tool for testing the software stack quickly and in an isolated manner. However, learning to work with Gazebo required a steep learning curve and a significant amount of additional effort to set up a realistic environment, including custom worlds and interactive objects.

When transferring the code base from the simulation environment to the real world, several problems occurred. The code had to be adjusted, for example by avoiding `use_sim_time`, and the robot's sensors were found to provide less accurate and less reliable feedback than expected.

5.3 The MIRTE Master Platform

The MIRTE Master proved to be a viable starting platform for a cleaning robot. Much of the required setup was already in place: MoveIt was available, Nav2 for MIRTE was present, Gazebo already existed, and many of the configuration files and calibrations were operational.

However, some parts of the design were not tested as thoroughly as expected before integration into the MIRTE Master platform. The most significant issues were found in the perception components. The RGB-D camera was initially mounted near the front and lower part of the robot, which resulted in depth images that were not suitable for point-cloud processing. The included USB camera module was also excluded from the final design because of poor performance, including motion blur and a pronounced drop in frame rate under changing lighting conditions.

The final major issue encountered was time synchronization between the MIRTE Master and other machines.

The Orange Pi 3B on the MIRTE Master was not powerful enough to run the full software stack on its own, so much of the processing had to be offloaded to a laptop. For this to work correctly, the time needed to be synchronized between the devices. Later, a custom implementation based on [Chrony](#) was used to synchronize the clocks automatically within acceptable tolerances.

In addition, the implemented navigation and perception approaches were feasible but not yet fully optimized. The coverage planners traded off completeness against path smoothness, and the perception pipeline remained sensitive to lighting and object appearance. These limitations provide useful guidance for future work.

6 Conclusion

This work has demonstrated that the different coverage planners each have distinct strengths and weaknesses. The morphology-based skeleton planner performed well in cluttered environments with tight spaces, while the spanning-tree planner provided better coverage in larger open spaces at the cost of more sharp turns and less accurate localization.

The developed perception pipeline successfully localized and classified small objects, demonstrating that this combination is an effective solution for object recognition on the MIRTE Master. The modifications to the manipulator also substantially improved positioning accuracy and grasp reliability, making objects manipulable even at less favorable approach angles.

Overall, this work has shown that the MIRTE Master can accommodate the components required for a laboratory-cleaning robot and is therefore a viable platform for this task. At the same time, the results also show that further improvements in sensing hardware, perception robustness, and testing procedures would be necessary before the system could be deployed more widely.

6.1 Future Work

Future research can use the software packages created as part of this work to evaluate the performance of the robot by testing the different coverage planners in isolation and in combination with the rest of the system. Additionally, other coverage planners, such as Voronoi-based [22] and boustrophedon [23] planners, can be implemented and tested. It could also be interesting to observe the influence of different environments on the overall performance of the system. Another possible avenue for future research is porting the MIRTE Master software stack to a more suitable single-board computer and observing differences in computational performance.

7 Appendix

7.1 Algorithms

Algorithm 1: Skeleton-Based Coverage Path Planning

```

Input: Occupancy grid  $\mathcal{M}$ , robot start position  $\mathbf{p}_0 \in \mathbb{R}^2$ 
Output: Ordered list of waypoint segments  $\mathcal{P} = [P_1, P_2, \dots, P_k]$ 
/* Map preprocessing */
 $B \leftarrow \text{threshold}(\mathcal{M}, \text{lethal\_threshold})$ ; // binary free-space mask
 $C \leftarrow \text{extractContours}(B)$ ; // OpenCV RETR_TREE
 $G_{poly} \leftarrow \text{groupContoursByContainment}(C)$ ; // outer + holes
foreach polygon group  $g \in G_{poly}$  do
     $S \leftarrow \text{medialAxis}(B)$ ; // skimage skeletonize
     $W \leftarrow \text{skeletonToWorldCoords}(S)$ ;
     $G_{graph} \leftarrow \text{buildKDTreeGraph}(W, \text{radius} = r_{path})$ ;
     $\text{bridgeDisconnectedComponents}(G_{graph})$ ;
     $L \leftarrow \text{findLeafNodes}(G_{graph})$ ; // nodes with degree 1
     $v \leftarrow \text{nearestLeaf}(\mathbf{p}_0, L)$ ;
     $\mathcal{Q} \leftarrow [v], V_{visited} \leftarrow \emptyset$ ;
    while  $V_{visited} \neq L$  do
         $V_{visited} \leftarrow V_{visited} \cup \{v\}$ ;
         $r \leftarrow \text{nearestLeafByGraphDistance}(v, L \setminus V_{visited}, G_{graph})$ ;
         $\pi \leftarrow \text{shortestPath}(v, r, G_{graph})$ ; // Dijkstra
        append  $\pi$  to  $\mathcal{Q}$ ;
         $v \leftarrow r$ ;
     $P_i \leftarrow [W[u] \forall u \in \mathcal{Q}]$ ;
    append  $P_i$  to  $\mathcal{P}$ ;
     $\mathbf{p}_0 \leftarrow P_i[\text{last}]$ ; // chain segments
sanitiseWaypoints( $\mathcal{P}, \mathcal{M}$ ); // clamp to costmap bounds
return  $\mathcal{P}$ 

```

Algorithm 2: Spanning-Tree Coverage Path Planning

Input: Occupancy grid $\mathcal{M}$, robot start position $\mathbf{p}_0 \in \mathbb{R}^2$, scale factor $s = 0.06$
Output: Ordered list of waypoint segments $\mathcal{P} = [P_1, P_2, \dots, P_k]$

```

/* Map preprocessing */
B ← threshold( $\mathcal{M}$ , lethal_threshold) ; // binary free-space mask
Bdown ← resize(B, scale = s, interpolation = NEAREST) ; // downsample
/* Build spanning tree over free cells */
G ← gridGraph(Bdown) ; // 4-connected grid graph
foreach edge (u, v) ∈ G do
    if Bdown[u] = 0 or Bdown[v] = 0 then
        remove (u, v) from G;
T ← ∅;
foreach connected component C ∈ G do
    T ← T ∪ DFS_tree(C);
/* Edge subdivision | insert midpoint nodes */
foreach edge (u, v) ∈ T do
    m ←  $\frac{u+v}{2}$ ;
    replace (u, v) with (u, m) and (m, v);
/* Generate circumnavigation contours */
Pperim ←  $\mathbf{0}^{H \times W}$  ; // blank perimeter map
r ←  $\lfloor \frac{1}{4s} \rfloor$  ; // rectangle half-size in pixels
foreach node p ∈ T do
    pimg ←  $\frac{p+0.5}{s}$  ; // scale back to image coords
    drawFilledRect(Pperim, centre = pimg, halfsize = r);
C ← findContours(Pperim, RETR_EXTERNAL);
/* Convert contours to world-frame waypoints and plan */
P ← [];
foreach contour c ∈ C do
    W ← pixelToWorld(c);
    if |W| < 2 then
        continue;
    idx ← arg mini ||W[i] − p0||2 ; // start from nearest point
    Pi ← [W[idx], W[idx + 1], ..., W[-1], W[0], ..., W[idx − 1]];
    append Pi to P;
    p0 ← Pi[last];
sanitiseWaypoints(P, M);
return P

```

Algorithm 3: Coverage Task Execution with Pause/Resume

Input: Coverage segments $\mathcal{P}$, planner type τ
Output: Task result (success / cancelled / stopped)

```

 $\mathcal{P} \leftarrow \text{sortByLength}(\mathcal{P}, \text{descending});$ 
 $n \leftarrow |\mathcal{P}|, i \leftarrow 0;$ 
while  $\mathcal{P} \neq \emptyset$  do
   $P_i \leftarrow \text{dequeue}(\mathcal{P}), i \leftarrow i + 1;$ 
   $\text{nav2.goThroughPoses}(\text{toROSPath}(P_i));$ 
  repeat
     $f \leftarrow \text{nav2.getFeedback}();$ 
     $\text{publishFeedback}(i, n, f.\text{remainingPoses});$ 
    if cancelRequested then
       $\text{nav2.cancelTask}();$ 
      return CANCELLED;
    if stopRequested then
       $\text{nav2.cancelTask}();$ 
      return STOPPED;
    if pauseRequested then
       $\text{nav2.cancelTask}();$ 
       $P_{rem} \leftarrow \text{computeRemainingSegment}(P_i, \text{getRobotPos}());$ 
       $\text{prepend } P_{rem} \text{ to } \mathcal{P};$                                      // resume from current pose
      repeat
         $\text{spinOnce}();$ 
      until not pauseRequested;
      ;
       $\text{nav2.goThroughPoses}(\text{toROSPath}(P_{rem}));$ 
  until  $\text{nav2.isTaskComplete}();$ 
   $\text{publishFeedback}(i, n, 0);$ 
return SUCCESS

```

Algorithm 4: Depth-Based Object Detection

Input: Point cloud $\mathcal{C}$ (camera frame), costmap $\mathcal{M}$, TF tree
Output: Detected object array $\mathcal{O}$

$\mathcal{C} \leftarrow \text{removeNaN}(\mathcal{C});$
 $\mathcal{C}_{LQ} \leftarrow \mathcal{C}, \text{mask} \leftarrow \mathbf{1}^{|\mathcal{C}|};$
 /* Iterative RANSAC plane removal */
repeat
 $(\mathbf{n}, d) \leftarrow \text{RANSAC_segmentPlane}(\mathcal{C}_{LQ},$
 $\epsilon = 0.003,;$
 $n_{iter} = 2000);$
 $\mathcal{C}_{LQ} \leftarrow \mathcal{C}_{LQ} \setminus \text{inliers};$
 $\text{mask} \leftarrow \text{mask} \wedge (\text{dist}(\mathcal{C}, \Pi) > 0.008);$
until $|\mathcal{C}_{LQ}| < 100$ **or** $|\text{inliers}| < 500;$
 $\mathcal{C}_{obj} \leftarrow \mathcal{C}[\text{mask}];$
 /* Coordinate transform to map frame */
 $T_{cam \rightarrow base} \leftarrow \text{TF.lookup}(\text{base_link}, \text{camera_frame});$
 $T_{base \rightarrow map} \leftarrow \text{TF.lookup}(\text{map}, \text{base_link});$
 $\mathcal{C}_{map} \leftarrow T_{base \rightarrow map} \cdot T_{cam \rightarrow base} \cdot \mathcal{C}_{obj};$
 /* Costmap filtering */
 $\mathcal{C}_{map} \leftarrow \{\mathbf{p} \in \mathcal{C}_{map} : \mathcal{M}(\mathbf{p}) < \delta_{occ}\};$
 /* Clustering and bounding box extraction */
 $L \leftarrow \text{DBSCAN}(\mathcal{C}_{map}, \epsilon = 0.03, \text{minPts} = 20);$
 $\mathcal{O} \leftarrow [];$
foreach cluster $k \in L, k \neq -1$ **do**
 $OBB_k \leftarrow \text{getOrientedBoundingBox}(\mathcal{C}_{map}[L=k]);$
 if $\mathcal{M}(OBB_k.\text{center}) < \delta_{occ}$ **then**
 $\psi \leftarrow \text{arctan2}(R_k[1, 0], R_k[0, 0]);$
 append $\text{DetectedObject}(OBB_k.\text{center}, OBB_k.\text{extent}, \psi)$ to $\mathcal{O};$ // yaw only
return $\mathcal{O}$

7.2 Materials

7.2.1 Hardware Specifications

Most of the hardware used in this implementation of the MIRTE Master comes from the standard components provided by the MIRTE team. The list of hardware parts, which excludes only trivial components such as nuts and bolts, can be found on the [Materials](#) web page. Given that MIRTE is not produced in large quantities and no standardized spare parts are available, 3D-printing was a practical way to produce the required structural parts from adjusted versions of the provided CAD models. These parts can be divided into two groups: chassis and manipulator.

7.2.2 Chassis

The chassis consists of a top, bottom, and manipulator-mounting plate, for which aluminium plates with a thickness of 1.5 mm were used, as well as several side panels. These side panels not only act as chassis support elements but also serve as mounting points for several electronic components. For these purposes, PETG was chosen because it is easy to print, impact-resistant, and sufficiently stiff for the intended application. The aluminium plates were chosen for their rigidity and durability. Although no formal material tests were performed in this study, these choices were made to match the practical requirements of the robot's payload and expected handling.

The translucent PETG variant used for this project also enables the user to look at the status lights on the inside of the chassis. This would otherwise not have been possible due to the aluminium top and bottom plates. During printing, orange PETG accent lines were also added to make the robot more visible to people passing by.

7.2.3 Manipulator

The manipulator consists of several components, which are listed on the [Materials](#) web page. This system can be divided into four groups: brackets, limbs, servo-motors, and the gripping mechanism. Given that there are four servo-motors, the arm is defined to have four degrees of freedom to be able to reach all places around the robot.

There is a fifth servo-motor mounted, but it only actuates the gripping mechanism and therefore does not add any degree of freedom to the system. All of this, including the chassis, can be seen in the interactive display near the bottom of the [Mechanical](#) overview page.

7.2.4 Software specifications

The MIRTE Master robotic platform has free open-source software available for any user to install.

The latest stable release is used in this robotic system. This software comes packaged inside a ROS application [24], a standardized framework for developing distributed robotic systems. This allows for the use of a wide variety of cross-compatible plugins and additional software packages, making rapid prototyping and development more efficient.

The ROS distribution used by the MIRTE Master platform is ROS 2 Humble Hawksbill on Ubuntu 22.04.

SLAM Before the robot is able to perform any complex task in an environment, the environment must first be mapped. For this, SLAM (Simultaneous Localization and Mapping) is used. This way, the robot can dynamically update its map of the environment based on measurements from its LiDAR scanner. The robot creates an occupancy grid map, represented as an image, while estimating its own pose. The two most widely adopted SLAM frameworks in the ROS ecosystem are Cartographer [25] and SLAM Toolbox [6]. For the MIRTE Master platform, SLAM Toolbox was selected due to its superior mapping accuracy and localization performance when reliable odometry is available [26]. Since the MIRTE Master is dedicated to running the core robotics software stack and is not burdened by other computationally intensive workloads, the higher computational requirements of SLAM Toolbox do not pose a significant limitation. Consequently, prioritizing mapping accuracy over computational efficiency makes SLAM Toolbox the most suitable choice for this application. SLAM Toolbox was also chosen for its native ROS 2 support and good integration with other ROS 2 packages. SLAM Toolbox allows for straightforward modification of mapping behavior using the set of parameters it provides. In the case of the MIRTE Master, there are several projects that have implemented SLAM Toolbox, so finding a ready-to-use set of SLAM parameters for this robot is relatively simple. The [Mirte Navigation](#) ROS package is used as a starting point for configuration files and SLAM parameter settings. It was assumed that the robot would only navigate in unknown environments without predefined or recurring maps. Localization (using AMCL) was therefore disabled in the application.

Manipulation While the arm on the MIRTE Master can be controlled in joint-space, the implementation relies on task-space (Cartesian-space) control over the end-effector position.

The manner in which motion planners are typically implemented requires the integration of several complex subsystems, such as inverse and forward kinematics solvers, trajectory planners, and path planners. To

accomplish the goal of task-space control over the arm, Movelt 2 ([27]) was used.

Navigation Robot navigation poses a challenge analogous to that of robot manipulation: workspace control. The robot must be able to navigate to or through a

set of waypoints given in the coordinate system of the map, while also implementing a real-time controller for obstacle avoidance. Similar to Movelt 2, Nav2 is used as a solution to this problem ([7]). Aside from path planning and real-time control, Nav2 also provides a [Costmap](#).

7.3 Figures

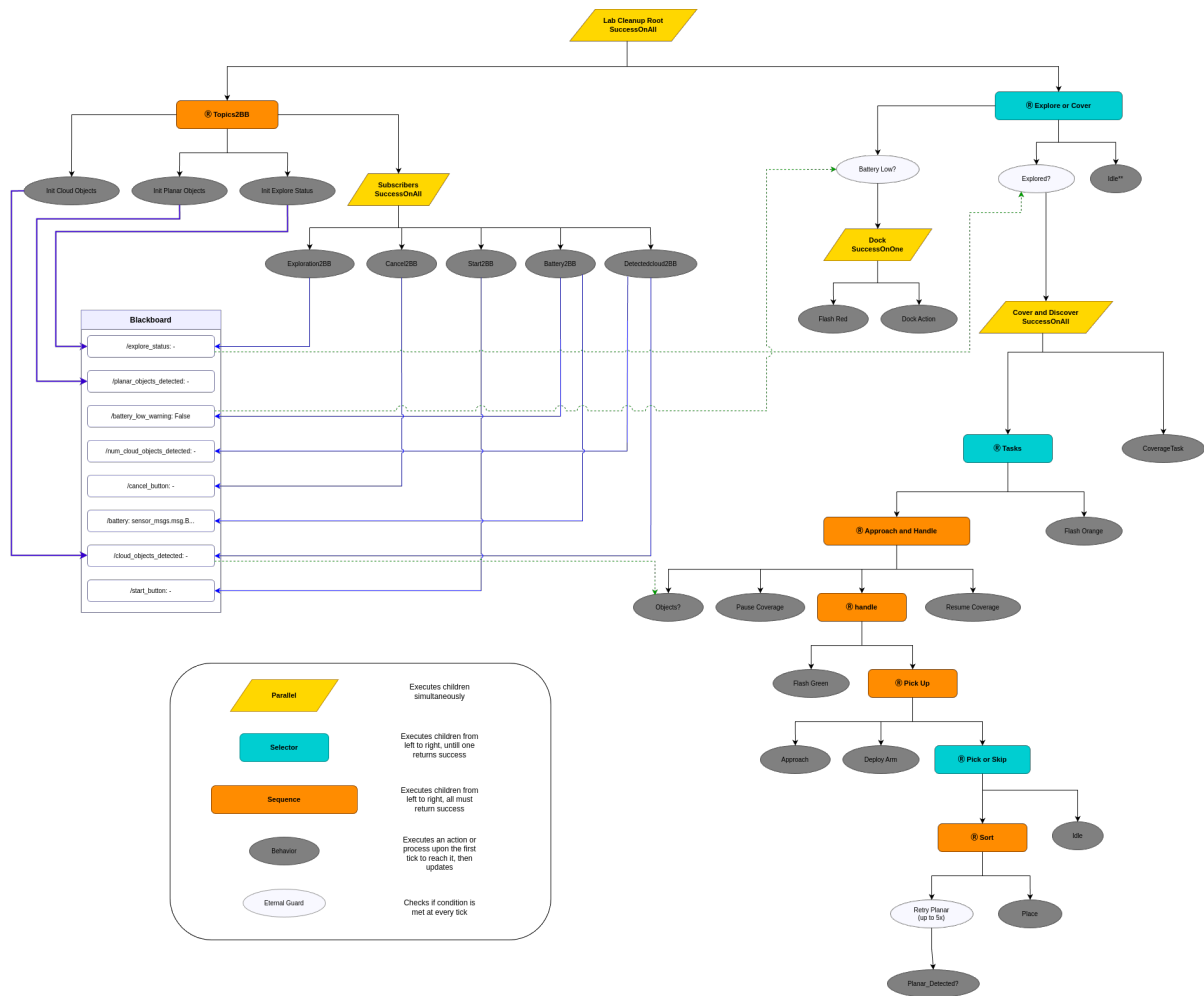


Figure 28: Full Behavior Tree

References

- [1] A. J. Becoy, K. Khomenko, L. Peternel, and R. T. Rajan, "Autonomous navigation of quadrupeds using coverage path planning with morphological skeleton maps", *Frontiers in Robotics and AI*, vol. Volume 12 - 2025, 2025, ISSN: 2296-9144. DOI: [10.3389/frobt.2025.1601862](https://doi.org/10.3389/frobt.2025.1601862). [Online]. Available: <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1601862>.
- [2] Y. Gabriely and E. Rimon, "Spanning-tree based coverage of continuous areas by a mobile robot", in *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No.01CH37164)*, vol. 2, 2001, 1927–1933 vol.2. DOI: [10.1109/ROBOT.2001.932890](https://doi.org/10.1109/ROBOT.2001.932890).
- [3] J. R. Sánchez-Ibáñez, C. J. Pérez-del-Pulgar, and A. García-Cerezo, "Path planning for autonomous mobile robots: A review", *Sensors*, vol. 21, no. 23, 2021, ISSN: 1424-8220. DOI: [10.3390/s21237898](https://doi.org/10.3390/s21237898). [Online]. Available: <https://www.mdpi.com/1424-8220/21/23/7898>.
- [4] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson, "From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2", *Robotics and Autonomous Systems*, vol. 168, p. 104493, 2023. DOI: [10.1016/j.robot.2023.104493](https://doi.org/10.1016/j.robot.2023.104493).
- [5] J. Stücker, R. Steffens, D. Holz, and S. Behnke, "Efficient 3d object perception and grasp planning for mobile manipulation in domestic environments", *Robotics and Autonomous Systems*, vol. 61, no. 10, pp. 1106–1115, 2013, Selected Papers from the 5th European Conference on Mobile Robots (ECMR 2011), ISSN: 0921-8890. DOI: <https://doi.org/10.1016/j.robot.2012.08.003>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0921889012001297>.
- [6] S. Macenski and I. Jambrecic, "Slam Toolbox: Slam for the dynamic world", *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021. DOI: [10.21105/joss.02783](https://doi.org/10.21105/joss.02783). [Online]. Available: <https://doi.org/10.21105/joss.02783>.
- [7] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A Navigation System", in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. [Online]. Available: <https://github.com/ros-planning/navigation2>.
- [8] H. Niu, X. Ji, L. Zhang, F. Wen, R. Ying, and P. Liu, *A Skeleton-Based Topological Planner for Exploration in Complex Unknown Environments*, <https://arxiv.org/abs/2412.13664>, 2025. [Online]. Available: <https://arxiv.org/abs/2412.13664>.
- [9] G. Williams, A. Aldrich, and E. A. Theodorou, "Model Predictive Path Integral Control: From Theory to Parallel Computation", *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017.
- [10] S. M. Elbhouy, A. Adamanov, P. M. Braun, and H. W. Rose, *Comparative Analysis of Local Trajectory Planning Algorithms in ROS2*, Unpublished manuscript, 2024.
- [11] S. Thrun, D. Fox, and W. Burgard, "The dynamic window approach to collision avoidance", *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
- [12] S. Macenski, S. Singh, F. Martin, and J. Ginés Clavero, "Regulated Pure Pursuit for Robot Path Tracking", *Autonomous Robots*, vol. 47, no. 6, pp. 685–694, 2023. DOI: [10.1007/s10514-023-10097-6](https://doi.org/10.1007/s10514-023-10097-6).
- [13] PickNik Robotics, *Moveit 2 API Documentation*, <https://moveit.picknik.ai/main/api/html/index.html>, Accessed: 2026-06-11, 2025. [Online]. Available: <https://moveit.picknik.ai/main/api/html/index.html>.
- [14] MIRTE, *Moveit Tutorial*, <https://docs.mirte.org/develop/doc/tutorials/moveit.html>, Accessed: 2026-06-11, 2025. [Online]. Available: <https://docs.mirte.org/develop/doc/tutorials/moveit.html>.
- [15] Ultralytics, *Ultralytics YOLO26*, <https://docs.ultralytics.com/models/yolo26/>, Accessed: 2026-06-11, 2026. [Online]. Available: <https://docs.ultralytics.com/models/yolo26/>.
- [16] R. Sapkota, R. H. Cheppally, A. Sharda, and M. Karkee, "Yolo26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection", 2026. [Online]. Available: <https://arxiv.org/abs/2509.25164>.
- [17] A. Limaran, A. Wicaksono, and P. Herwanto, "Performance enhancement of embedded object detection via neural hardware acceleration", *TELKOMNIKA (Telecommunication Computing Electronics and Control)*, vol. 24, p. 126, Feb. 2026. DOI: [10.12928/telkomnika.v24i1.27448](https://doi.org/10.12928/telkomnika.v24i1.27448).
- [18] R. Sapkota and M. Karkee, "Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 Object Detectors for Computer Vision and Pattern Recognition", 2026. [Online]. Available: <https://arxiv.org/abs/2510.09653>.

- [19] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3d: A modern library for 3d data processing", *arXiv preprint arXiv:1801.09847*, 2018.
- [20] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography", *Commun. ACM*, vol. 24, no. 6, pp. 381–395, Jun. 1981, ISSN: 0001-0782. DOI: [10.1145/358669.358692](https://doi.org/10.1145/358669.358692). [Online]. Available: <https://doi.org/10.1145/358669.358692>.
- [21] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise", in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, ser. KDD'96, AAAI Press, Portland, Oregon, 1996, pp. 226–231.
- [22] Q. Du, V. Faber, and M. Gunzburger, "Centroidal Voronoi Tessellations: Applications and Algorithms", *SIAM Review*, vol. 41, pp. 637–676, Aug. 2006. DOI: [10.1137/S0036144599352836](https://doi.org/10.1137/S0036144599352836).
- [23] H. Choset and P. Pignon, "Coverage Path Planning: The Boustrophedon Cellular Decomposition", in *Proceedings of 1st International Conference on Field and Service Robotics (FSR '97)*, 1997, pp. 216–222.
- [24] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild", *Science Robotics*, vol. 7, no. 66, eabm6074, 2022. DOI: [10.1126/scirobotics.abm6074](https://doi.org/10.1126/scirobotics.abm6074). [Online]. Available: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [25] P. Kicman, P. Silson, A. Tsourdos, and A. Savvaris, "Cartographer - A Terrain Mapping and Landmark-based Localization System for Small Robots", in *AIAA Infotech at Aerospace Conference and Exhibit 2011*, Mar. 2011, ISBN: 978-1-60086-944-0. DOI: [10.2514/6.2011-1513](https://doi.org/10.2514/6.2011-1513).
- [26] İ. İnce, D. Yılmaz-Kaplan, and F. Keleş, "From simulation to reality: Comparative performance analysis of slam toolbox and cartographer in ros 2", *Electronics*, vol. 14, no. 24, 2025, ISSN: 2079-9292. DOI: [10.3390/electronics14244822](https://doi.org/10.3390/electronics14244822). [Online]. Available: <https://www.mdpi.com/2079-9292/14/24/4822>.
- [27] D. Coleman, I. A. Sucan, S. Chitta, and N. Correll, "Reducing the Barrier to Entry of Complex Robotic Software: A MoveIt! Case Study", *Journal of Software Engineering for Robotics*, vol. 5, no. 1, pp. 3–16, 2014. DOI: [10.6092/JOSER_2014_05_01_p3](https://doi.org/10.6092/JOSER_2014_05_01_p3).